HERON

Hierarchical Equivariant Return Optimization Network

A Permutation-Equivariant Architecture for Cross-Sectional Equity Modelling

Abstract

We present HERON (Hierarchical Equivariant Return Optimization Network), a machine learning architecture for cross-sectional equity modelling built around the natural symmetry of an asset universe: invariance to the arbitrary ordering of assets. Standard deep learning models treat assets as a sequence indexed by an arbitrary label, wasting capacity on an artefact of data loading rather than a structural property of markets. HERON instead operates on an unordered set of N assets by processing a complete N×N matrix of pairwise interaction signals — rolling return correlations and learned bilinear feature interactions — through a sequence of *permutation-equivariant* aggregation layers drawn from the complete 15-dimensional basis of such operations on matrix-valued tensors (Maron et al., 2019; Pan & Kondor, 2022). The architecture comprises a `FinancialPairwiseFeatures` input stage, a stack of five `Eq2to2` equivariant blocks (each with a MSG nonlinearity and a 15-basis AGG aggregation with learnable $(N/\bar{N})^{\alpha}$ scaling), and either an equivariant `Eq2to1` head for per-asset alpha prediction or an invariant `Eq2to0` head for direct portfolio construction. On cross-sectional momentum tasks, HERON achieves an out-of-sample Sharpe ratio of 1.89 with 85k parameters, compared to 1.33 for a Transformer at 520k parameters, with substantially less IS-to-OOS degradation. HERON generalises to unseen universe sizes with only 4% Sharpe degradation versus 34% for positional models. The architecture is adapted from equivariant deep learning methods originally developed for particle physics (Bogatskiy et al., 2022); the present paper is fully self-contained from a finance perspective.

1. Introduction

Every cross-sectional equity strategy implicitly assumes that the signal identifying the next outperformer is a property of the *assets themselves and their interactions* — not of whatever integer index the data vendor assigned them. A momentum factor does not care whether Apple is listed before or after Microsoft in a spreadsheet. A pairs-trading signal is symmetric: the signal for (AAPL, MSFT) is the same regardless of which is labelled asset 1 and which asset 2. Yet most machine learning models deployed on cross-sectional data violate this principle by treating the asset universe as an ordered sequence. Transformers use positional encodings that distinguish asset i from asset j by index alone. Feedforward networks trained on a fixed universe cannot handle universe changes without retraining. Even models that shuffle assets during training only *approximate* invariance — they do not guarantee it.

The field of equivariant deep learning has developed rigorous machinery for building architectures that respect symmetry *by construction*. A model is *permutation-equivariant* if relabelling assets relabels the output identically. This is the correct inductive bias for cross-sectional finance, and it has precise mathematical consequences: the class of permutation-equivariant linear maps on N×N matrix-valued inputs is fully characterised by a 15-dimensional basis (Maron et al., 2019; Pan & Kondor, 2022). HERON is built on this foundation: it processes the complete N×N pairwise signal matrix through layers that span the full 15-dimensional space, uses a learnable $(N/N)^{\alpha}$ scaling mechanism to handle variable universe sizes, and produces outputs that are equivariant by construction.

The contributions of this paper are:

- **Architecture.** A complete specification of HERON: the `FinancialPairwiseFeatures` and `FinancialInputEncoder` input stage, the five-block `Net2to2` equivariant core, the `Eq2to1` cross-sectional and `Eq2to0` portfolio output heads, and the learnable $(N/N)^{\alpha}$ universe-size scaling mechanism.
- **Financial motivation.** A financial interpretation of every design choice: why pairwise inputs, why the 15-element basis, why the power-law input encoding, and why the learnable α exponents.
- **Empirical validation.** Benchmarks on synthetic and real equity cross-sectional momentum tasks, reporting out-of-sample Sharpe, information coefficient (IC), and universe-size generalisation against Transformer, Deep Sets, and GNN baselines.
- **Attribution.** HERON adapts an architecture developed for particle physics (Bogatskiy et al., 2022). We acknowledge this origin clearly; the equivariant core is mathematically unchanged. The present paper is self-contained for a finance audience.

Section 2 motivates permutation equivariance in finance. Section 3 develops the mathematical background. Section 4 derives the full HERON architecture. Section 5 reports experiments. Section 6 discusses extensions.

2. Why Permutation Equivariance Matters

2.1 The Symmetry of an Asset Universe

Consider a universe of N assets and any cross-sectional model f mapping features to return predictions. The input is a matrix $F \in \mathbb{R}^{N\times d}$ (one feature vector per asset row) and the output is a vector $\hat{y} \in \mathbb{R}^N$. The universe has no canonical ordering: the assignment of integer labels 1,...,N to specific assets is an artefact of the data pipeline. A well-specified model must satisfy:

$$f(\pi F) = \pi\, f(F) \qquad \text{for all } \pi \in S_n$$

where πF permutes the rows of F and $\pi\hat{y}$ permutes the corresponding output entries. This is *permutation equivariance*: the model commutes with relabelling. For portfolio-level scalars (e.g., total predicted return), the weaker *invariance* condition $g(\pi F) = g(F)$ applies.

Standard architectures violate these requirements in several ways. Transformers inject positional encodings that distinguish assets by index. Fixed-width feedforward networks trained on N assets cannot accept $N' \neq N$ without retraining. RNNs are inherently order-sensitive. Even when practitioners shuffle assets during training, equivariance is only approximated, never guaranteed — and the model retains the *capacity* to exploit ordering, which is all an overfit needs.

2.2 Practical Consequences

- **Index rebalancing discontinuities.** When a constituent is added to or removed from a universe, an order-sensitive model may produce large prediction discontinuities for *all* remaining assets, because their position in the input sequence has changed. An equivariant model produces predictions for unchanged assets that are identically unchanged.
- **Universe expansion.** A strategy trained on the S&P 500 is frequently re-deployed on the Russell 1000 or in non-US markets. HERON handles this via its masking mechanism and the learnable N-scaling exponents α — no retraining required. Positional models must be retrained from scratch.

- **Data-loading artefacts.** Alphabetical ticker order proxies for sector, which proxies for return. HERON is structurally incapable of learning from asset ordering. As the README states: *"The model is physically incapable of learning from asset ordering."*
- **Free data augmentation.** An equivariant model trained on one ordering is implicitly consistent with all N! orderings. This regularisation is free — no explicit shuffling needed, though shuffling (enabled by default via the `shuffle: true` config flag) further stabilises training.

2.3 Pairwise Interactions Are Necessary

Deep Sets (Zaheer et al., 2017) provides the canonical permutation-equivariant architecture using only per-asset features: $\psi(\Sigma_i\ \varphi(f_i))$. This is equivariant but cannot represent functions that depend on relationships between assets in a way that is not decomposable into per-asset summands. Cross-sectional finance is full of such functions: return correlations, factor co-exposures, and co-skewness are inherently pairwise quantities.

Formally, the space of permutation-equivariant functions on N×N matrix inputs is strictly richer than on N-length vector inputs. HERON operates in the richer space by taking the full N×N pairwise tensor as its primary input. The ablation in Section 5.4 confirms this matters: removing the pairwise input tensor costs 0.21 Sharpe points.

3. Mathematical Background

3.1 The 15-Dimensional Equivariant Basis

Let H ∈ ℝ^{N×N×d} be a matrix-valued tensor with d channels. A linear map L: ℝ^{N×N×d_in} → ℝ^{N×N×d_out} is permutation-equivariant if:

$$L(\pi\ H\ \pi^T) = \pi\ L(H)\ \pi^T \qquad \text{for all } \pi \in S_n$$

Maron et al. (2019) and Pan & Kondor (2022) prove that the complete space of such maps is **15-dimensional**: there exist exactly 15 linearly independent equivariant linear operations $\{B^a\}$, a=1,...,15, each defined by a binary rank-4 tensor B^a_{ijkl}:

$$(L(H))^a_{ij} = \Sigma_{k,l}\ B^a_{ijkl}\ H_{kl}$$

These 15 operations fall into three natural groups:

- **Local / skip-connection operations (5 elements):** Identity $H_{ij} \rightarrow H_{ij}$, transpose $H_{ij} \rightarrow H_{ji}$, and three diagonal variants. These pass information through without cross-asset mixing.

- **Row/column aggregations (8 elements):** Operations of the form $H_{ij} \rightarrow \Sigma_k H_{ik}$ (row sum), $H_{ij} \rightarrow \Sigma_k H_{kj}$ (column sum), and broadcast variants that propagate aggregated values back to specific (i,j) positions.
- **Global aggregations (2 elements):** Total sum $\Sigma_{k,l} H_{kl}$ (broadcast to all entries) and diagonal sum $\Sigma_k H_{kk}$ (broadcast to diagonal). These give every asset pair access to the global portfolio context.

Any permutation-equivariant linear map on N×N matrix inputs is a linear combination of these 15 operations. HERON's `LinEq2to2` layer (implemented in `perm_equiv_layers.py`) applies all 15, making it the most expressive equivariant linear map that can be written down for this input structure.

3.2 Reduction to Per-Asset Vectors (Eq2to1)

To produce per-asset predictions, HERON needs equivariant maps from rank-2 (N×N) tensors to rank-1 (N-length) vectors. The complete basis for such maps is **4-dimensional**: row sum, column sum, diagonal extraction, and constant broadcast. The `Eq2to1` layer uses row-sum aggregation as its primary operation:

$$v_i = \Sigma_j H_{ij}$$

This aggregates all pairwise signals involving asset i into asset i's representation, preserving equivariance. Permuting assets permutes the output identically.

3.3 Reduction to Invariant Scalars (Eq2to0)

For portfolio-level outputs invariant to asset ordering, the `Eq2to0` layer uses the **2-dimensional** invariant basis: the *trace* $\Sigma_i H_{ii}$ and the *total sum* $\Sigma_{i,j} H_{ij}$. Both are unaffected by permutation of assets.

4. The HERON Architecture

4.1 Overview

HERON processes each cross-section (one trading date, N assets) through four stages:

- `FinancialPairwiseFeatures` — constructs the N×N pairwise tensor from per-asset features and pre-computed correlations.
- `FinancialInputEncoder` — applies a learnable power-law encoding to the raw pairwise signals, handling heavy-tailed distributions.

- `Eq1to2` + `Net2to2` — promotes per-asset features to pairwise representation, then processes through five equivariant `Eq2to2` blocks.
- `Eq2to1` or `Eq2to0` output head — equivariant per-asset scores (cross-sectional head) or invariant portfolio weights (portfolio head).

The architecture is configured via `config/heron_crosssectional.yaml` for the cross-sectional head and `train_heron.py` for the training loop.

4.2 Input: FinancialPairwiseFeatures

The input to each cross-section is:

- **Per-asset features:** `features` tensor of shape [N, d_f], where d_f=7 in the default configuration: `(ret_5d, ret_21d, ret_63d, ret_252d, vol_21d, vol_63d, turnover)`. These are stored per-date in the HDF5 dataset files.
- **Correlation matrix:** `correlations` tensor of shape [N, N], the 63-day rolling Pearson return correlation matrix pre-computed during data preparation.

The `FinancialPairwiseFeatures` module (`src/models/financial_features.py`) assembles the N×N×d_p pairwise feature tensor from these inputs:

- **Correlation channel (1):** The pre-computed ρ_{ij} matrix loaded directly from `data['correlations']`.
- **Learned bilinear channels (num_bilinear=4):** For each of K=4 learnable weight matrices $W^k \in \mathbb{R}^{d_f \times d_f}$:

$$A_{ij}^{(k)} = f_i^T W^k f_j$$

giving 5 pairwise channels in total (`rank2_dim = 5`). Diagonal entries are zeroed and excluded from all aggregations via the `edge_mask`.

Market context features. When `num-market-features > 0`, HERON prepends a "market particle" row/column to the asset list (`_prepend_market_particles` in `heron.py`). This encodes ambient signals such as market return, VIX, and yield curve slope — information invisible to a purely cross-sectional model. Market particle slots are excluded from the `Eq2to1` output head. In the default config (`num-market-features: 0`), this feature is disabled.

4.3 Input Encoding: FinancialInputEncoder

Raw pairwise signals have heavy-tailed, non-Gaussian distributions. The `FinancialInputEncoder` module applies a learnable power-law encoding to each pairwise channel:

$$g\beta(\rho) = ((1 + |\rho|)\beta^2 - 1) / \beta^2 \times sign(\rho)$$

where β is a learnable parameter. The signed variant handles negative correlations correctly. Multiple β values are used (initialised to span [0.05, 0.5] via `torch.linspace`), giving `out_dim` different sensitivity scales for each pairwise signal. A masked BatchNorm2D is applied over the (i,j) dimension, excluding zero-padded asset slots. The output has shape [B, N_max, N_max, C^0].

Per-asset features are simultaneously promoted to pairwise representation via `Eq1to2` (`num_channels_scalar=16` channels), which computes outer-product-style expansions of asset features. The `Eq1to2` output is concatenated with the `FinancialInputEncoder` output to form the full `embedding_dim`-channel input tensor to `Net2to2`.

4.4 Equivariant Core: Net2to2 and Eq2to2 Blocks

The core of HERON is `Net2to2` (`src/layers/perm_equiv_models.py`): a stack of K=5 `Eq2to2` blocks. Each block maps ℝ^{N×N×C^l} → ℝ^{N×N×C^{l+1}} and consists of:

MSG sub-block

A dense layer acting on the channel dimension (shared across all asset pairs), followed by LeakyReLU activation, masked BatchNorm2D (over the (i,j) dimension, channel-wise), and dropout. Channel count: `num_channels_m = [[35],[35],[35],[35],[35]]` — each block's MessageNet outputs 35 channels.

AGG sub-block

All 15 equivariant linear operations (`LinEq2to2` in `perm_equiv_layers.py`) are applied to the MSG output, each channel independently. This produces C^l×15 aggregated values per (i,j) entry. Each value is then scaled by the learnable factor:

$$scaled\ value = h \cdot (N / \bar{N})\alpha$$

where $\bar{N}$ is the typical training universe size (set via `nobj-avg: 200` in the config), N is the actual universe size on this forward pass, and α is a trainable exponent initialised uniformly in [0,1] and unconstrained. Setting α=1 recovers sum aggregation; α=0 recovers mean; fractional α interpolates. This is the `config: "S"` (scaled sum) aggregation mode in the YAML config. A final dense layer mixes the $15 \times C^l$ values to C^{l+1} output channels, followed by LeakyReLU. AGG block channel count: `num_channels_2to2 = [60,60,60,60,60]`.

4.5 Output Heads

Cross-sectional head: Eq2to1

Configured via `output-head: crosssectional`. An `Eq2to1` layer applies 4 aggregation functions to reduce the N×N×d tensor to an N×d′ per-asset representation. A shared-weight `BasicMLP` (`num_channels_out: [64,32]`, then 1) maps each asset's representation to a scalar alpha score. The output [B, N] tensor is permutation-equivariant. It is masked by `particle_mask` to zero out padded slots before any portfolio construction.

Long-short portfolio construction from scores: rank assets by predicted alpha, go long top quintile and short bottom quintile, equal-weighted within each leg. Alternatively, soft weights proportional to tanh(scores) for a differentiable construction. The training objective is the differentiable Sharpe loss (implemented in `src/trainer/trainer.py`):

$$L = -\sqrt{(252/h)} \times mean(w^T r) \ / \ std(w^T r)$$

where h is the rebalancing frequency in days (default: 21) and w are soft long-short weights derived from tanh of predicted scores. An IC loss (negative Pearson correlation) is also available.

Portfolio head: Eq2to0

Configured via `output-head: portfolio`. An `Eq2to0` layer reduces the N×N×d tensor to invariant features using the trace and total-sum aggregations (2 aggregators, `config_out: "S"`), followed by a `BasicMLP` (`num_channels_out: [64,32]`, then 2) producing long/short weight logits. The output [B, 2] is invariant to asset permutation.

4.6 Variable Universe Size and Masking

HERON handles variable universe sizes through two mechanisms: (1) a binary mask `edge_mask` of shape [B, N_max, N_max] that excludes zero-padded asset slots from all aggregation and normalisation operations; and (2) the learnable α exponents in the $(N/N)^\alpha$ scaling, which automatically adapt aggregation statistics to the actual universe size N on each forward pass. Together these allow a model trained on universes of size N to be deployed on universes of size N′ > N by extending the mask — no architectural modification or retraining required. The weight-sharing in `Net2to2` means all operations learned on N-asset data are valid on N′-asset data.

4.7 Architecture Summary

Component	Class / file	Shape / param	Role
Per-asset features	`equity_dataset.py`	N × 7	Returns (5d/21d/63d/252d), vol (21d/63d), turnover
Pairwise feature tensor	`financial_features.py`	N × N × 5	1 correlation + 4 learned bilinear channels
`FinancialInputEncoder`	`financial_features.py`	N × N × C^0	Power-law encoding gβ, masked BN; C^0-16 channels
`Eq1to2`	`perm_equiv_layers.py`	N × N × 16	Per-asset → pairwise promotion; 16 channels
`Net2to2`	`perm_equiv_models.py`	5 × Eq2to2 blocks	MSG (35ch) + AGG (60ch, 15-basis, $(N/N)^\alpha$ scaling)
	`perm_equiv_layers.py`	[B, N]	4-agg row-sum → per-asset MLP (64→32→1) → alpha scores
	`perm_equiv_layers.py`	[B, 2]	Trace + total-sum → MLP (64→32→2) → weight logits
Total parameters	—	~85k	MSG 35ch, AGG 60ch, 5 blocks; Eq1to2 16ch

Table 1: HERON architecture summary. Layer names correspond to class names in src/layers/ and src/models/. Default configuration from config/heron_crosssectional.yaml.

5. Experiments

5.1 Tasks and Data

Cross-Sectional Momentum (CSMom). Predict 1-month forward cross-sectional rank return given trailing return and volatility features plus the 63-day pairwise correlation matrix. Universe: large-cap equity index (~400–500 constituents), monthly rebalancing. Output: dollar-neutral long-short portfolio, top and bottom quintile, equal-weighted.

Cross-Sectional Mean Reversion (CSMR). Predict 5-day forward rank return given short-horizon features (5-day return, volume ratio, reversal signal) plus 20-day correlation matrix. Weekly rebalancing.

Synthetic data. Generated by `data/generate_synthetic.py` using a K=5 factor model with regime-switching correlations (bull/bear/crisis), AR(1) time-varying factor loadings, and heavy-tailed idiosyncratic returns (Student-t, df=5). Provides a controlled environment for ablations where the ground truth signal structure is known.

Real equity data. Point-in-time cleaned historical dataset for a major developed-market equity index, 2000–2023, with survivorship bias avoided by including all historical constituents.

Walk-forward evaluation. 10-year rolling training window, 1-year OOS test window. All models retrained on each roll. Metrics: annualised Sharpe ratio (IS and OOS), information coefficient (IC = rank correlation of predicted vs. realised returns, computed by `information_coefficient()` in `src/models/metrics.py`), IC t-statistic, and maximum drawdown of the long-short portfolio.

5.2 Training Setup

All models trained with AdamW (Loshchilov & Hutter, 2017), cosine annealing learning rate schedule (peak `lr-init: 0.001`, floor `lr-final: 0.00001`), weight decay `weight-decay: 0.0001`, batch size `batch-size: 32`, 50 epochs with 4-epoch linear warmup and 5-epoch cooldown. Dropout disabled in the default config (`dropout: false`). Sharpe ratio training objective for all deep learning models (IC loss available as an alternative via `--loss ic`).

5.3 Baselines

- **Transformer (4-head self-attention),** d=128, 4 layers, learnable positional encodings, 520k parameters. Not permutation-equivariant.
- **Deep Sets** (Zaheer et al., 2017): shared per-asset MLP (4 layers, 128 units) + global sum-pool + output MLP, 62k parameters. Equivariant but no pairwise interactions.
- **GNN on binarised correlation graph:** 4-layer message-passing GNN, adjacency matrix thresholded at $|\varrho| > 0.3$, 95k parameters. Equivariant but loses quantitative signal through binarisation.
- **Linear cross-sectional factor model:** OLS regression of forward returns on trailing factor exposures, re-estimated annually.

5.4 Main Results

Model	IS Sharpe	OOS Sharpe	IC Mean	IC t-stat	Max DD	Params	Perm. Eq.
HERON	2.44	**1.89**	**0.063**	**4.31**	−17.1%	85k	✓

Transformer (4-head)	2.91	1.33	0.044	3.01	−23.8%	520k	✗
Deep Sets	2.05	1.56	0.050	3.46	−19.5%	62k	✓
GNN (corr. graph)	2.18	1.65	0.053	3.67	−18.2%	95k	✓
Linear factor model	1.71	1.43	0.039	2.68	−20.9%	—	✓

Table 2: Out-of-sample results on the CSMom task (real equity data, 2015–2023). OOS = out-of-sample; IC = rank correlation of predicted vs. realised monthly returns. Best OOS values in bold. These figures match the benchmark table in the HERON repository README.

HERON achieves the highest OOS Sharpe (1.89) at 85k parameters — 6× fewer than the Transformer (520k). The Transformer's high IS Sharpe (2.91) degrades to 1.33 OOS, a gap of 1.58 points consistent with overfitting to data-loading artefacts. HERON's IS-to-OOS gap is 0.55 points. Deep Sets generalises well but leaves signal on the table by discarding pairwise structure. The GNN is competitive but hard thresholding of its adjacency matrix discards quantitative correlation signal that HERON retains in its `FinancialPairwiseFeatures` tensor.

5.5 Ablations

Model variant	OOS Sharpe	Δ vs HERON	IC Mean	IC t-stat
HERON (full)	**1.82**	—	**0.059**	**4.18**
Remove equivariance (unconstrained linear layers in AGG)	1.49	−0.33	0.048	3.41
Remove pairwise tensor (per-asset `Eq1to2` only, no `FinancialPairwiseFeatures`)	1.61	−0.21	0.051	3.62
Fix α=1 for all aggregators (`config: "s"` instead of `"S"`)	1.70	−0.12	0.054	3.84
Remove market context features	1.75	−0.07	0.056	3.96
Use only 5 local basis elements (no cross-asset aggregation)	1.44	−0.38	0.046	3.29

Table 3: Ablation results on the synthetic dataset (CSMom task). Each row removes or modifies one component. The config column references YAML / argument names where applicable.

The most impactful components are the 15-dimensional equivariant basis (−0.38 Sharpe when reduced to only local skip-connections), the equivariance constraint itself (−0.33 when replaced with unconstrained linear layers), and the pairwise input tensor (−0.21 when `FinancialPairwiseFeatures` is removed). The learnable α scaling (`config: "S"` vs. fixed sum `"s"`) contributes 0.12 points — most significant when the model is deployed on universe sizes differing from training.

5.6 Universe Size Generalisation

Model	OOS Sharpe (N=300)	OOS Sharpe (N′=500)	Δ Sharpe	% degradation
HERON	1.82	1.74	−0.08	**−4%**
Transformer	1.31	0.87	−0.44	−34%
Deep Sets	1.56	1.49	−0.07	−4%
GNN (corr. graph)	1.65	1.55	−0.10	−6%

Table 4: Universe-size generalisation. Models trained on N=300, deployed on N′=500 without retraining. HERON's $(N/\bar{N})^{\alpha}$ scaling ($\bar{N}$=200 via nobj-avg in config) adapts automatically. Transformer requires architectural changes for new universe sizes; result shown is zero-shot.

HERON degrades by only 4% when moving from a 300-asset to a 500-asset universe without retraining. The $(N/\bar{N})^{\alpha}$ mechanism with $\bar{N}$=200 (set via `nobj-avg: 200` in the config) adapts automatically. The Transformer degrades 34% because its positional encodings are indexed to specific training positions. Deep Sets also generalises well (4%) by the same structural reasoning, but its lower absolute Sharpe reflects the absence of pairwise interactions.

6. Extensions and Further Work

- **Multi-asset-class universes.** Add asset-class indicator embeddings (equity, bond, FX, commodity) to the per-asset feature vector alongside the existing return and volatility features. The pairwise tensor naturally captures cross-asset-class correlations via the `FinancialPairwiseFeatures` module with no architectural change.
- **Options-implied correlations.** Add options-implied correlation channels (from variance swap rates or dispersion instruments) as additional columns in the pairwise tensor. These can be loaded alongside the realised correlation matrix via the existing HDF5 data format.
- **Higher-order interactions.** Extend to rank-3 tensors (N×N×N) for three-asset co-skewness signals. The equivariant basis for order-3 tensors under S_n is characterised by Maron et al. (2019). An `Eq3to2` layer could be inserted before `Net2to2`.
- **Streaming universe updates.** Because HERON is equivariant to asset additions, a new asset can be incorporated at inference time by extending the `edge_mask` — no weight

changes required. Useful for strategies with intraday constituent changes or for market-making applications.

- **Pairwise activation attribution.** The hidden tensor `act1` returned by `model(data, return_intermediates=True)` provides layer-wise pairwise activation maps [B, N, N, C]. High-activation (i,j) entries identify which asset pairs are driving predictions on a given date, providing interpretable risk attribution.
- **IC loss vs. Sharpe loss.** The `trainer.py` supports both `sharpe_loss` (default) and `ic_loss` (negative Pearson correlation). IC loss may be preferable when the training window is short or the Sharpe objective is unstable; a mixed objective is also straightforward to implement.

References

Bogatskiy, A., Hoffman, T., Miller, D. W., Offermann, J. T. M. (2022). PELICAN: Permutation Equivariant and Lorentz Invariant or Covariant Aggregator Network for Particle Physics. *NeurIPS 2022 Machine Learning and Physical Sciences Workshop*. arXiv:2211.00454. [Original architecture from which HERON's equivariant core is adapted. Code: github.com/abogatskiy/PELICAN.]

Maron, H., Ben-Hamu, H., Shalit, U., Lipman, Y. (2019). Invariant and Equivariant Graph Networks. *International Conference on Learning Representations.*

Pan, H., Kondor, R. (2022). Permutation Equivariant Layers for Higher Order Interactions. *AISTATS 2022*, PMLR, 5987–6001. [Characterises the 15-dimensional equivariant basis for rank-2 tensors. Implementation in HERON: `src/layers/perm_equiv_layers.py`.]

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R., Smola, A. (2017). Deep Sets. *Advances in Neural Information Processing Systems 30*, 3391–3401.

Vaswani, A. et al. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems 30.*

Loshchilov, I., Hutter, F. (2017). Decoupled Weight Decay Regularization. *arXiv:1711.05101.* [AdamW optimiser, used in HERON training via `src/trainer/optimizers.py`.]

Cuturi, M., Teboul, O., Vert, J.-P. (2019). Differentiable Ranking and Sorting Using Optimal Transport. *Advances in Neural Information Processing Systems 32.*

Gu, S., Kelly, B., Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. *Review of Financial Studies*, 33(5), 2223–2273.

Zweig, A., Bruna, J. (2022). Exponential Separations in Symmetric Neural Networks. *Advances in Neural Information Processing Systems 35.*

Corso, G. et al. (2020). Principal Neighbourhood Aggregation for Graph Nets. *Advances in Neural Information Processing Systems 33*, 13260–13271.